UE24CS341A SOFTWARE ENGINEERING · PES UNIVERSITY · JACKFRUIT MINI-PROJECT

Software Architecture and Design

Team 14 · Project 39: Dehallucination Add-On Module ("Dehallucinator")

Team member	Role
Sai Nikitha Perumalla (PES2UG24AM142)	Role A – Claim Extractor; Scrum Master
Shreya Raghuraj (PES2UG24AM154)	Role B – Evidence Retriever
Ralph D'souza (PES2UG24AM131)	Role C – Verifier & Scorer, testing, SonarQube
Satwik (PES2UG24AM148)	Role D – Corrector, API, UI, Jenkins, Docker

Repository: github.com/SoftwareEngineering-Runtime/dehallucinator · Jira project: DH
Date: 07 Nov 2026

Contents

1. High-Level Architecture (Role A – Sai Nikitha Perumalla)
2. Claim Extractor: Design (Role A – Sai Nikitha Perumalla)
3. Evidence Retriever: Architecture and Design (Role B – Shreya Raghuraj)
4. Verifier & Scorer: Architecture and Design (Role C – Ralph D'souza)
5. Corrector, Pipeline, API, UI and Deployment (Role D – Satwik)

Diagrams are UML class and sequence diagrams written in Mermaid in the repository (docs/architecture and docs/design) and rendered for this document.

1. High-Level Architecture (Role A – Sai Nikitha Perumalla)

Project 39: Dehallucination Add-On Module ("Dehallucinator")

1. High-Level Architecture Overview (Owner: Sai Nikitha Perumalla, Role A)

1.1 Architectural Style

Dehallucinator uses a **layered architecture** with a **pipe-and-filter** core. A request passes through four independent stages in a fixed order, and each stage takes the previous stage's output and adds to it.

- **Presentation:** the Streamlit demo chatbot and the `@dehallucinate` decorator for plugging into any chatbot.
- **Service:** a FastAPI REST API (`POST /check`, `GET /health`).
- **Core pipeline:** Claim Extractor → Evidence Retriever → Verifier & Scorer → Corrector.
- **Data and models:** spaCy language model, Wikipedia API, local knowledge base, NLI model, shared data models.

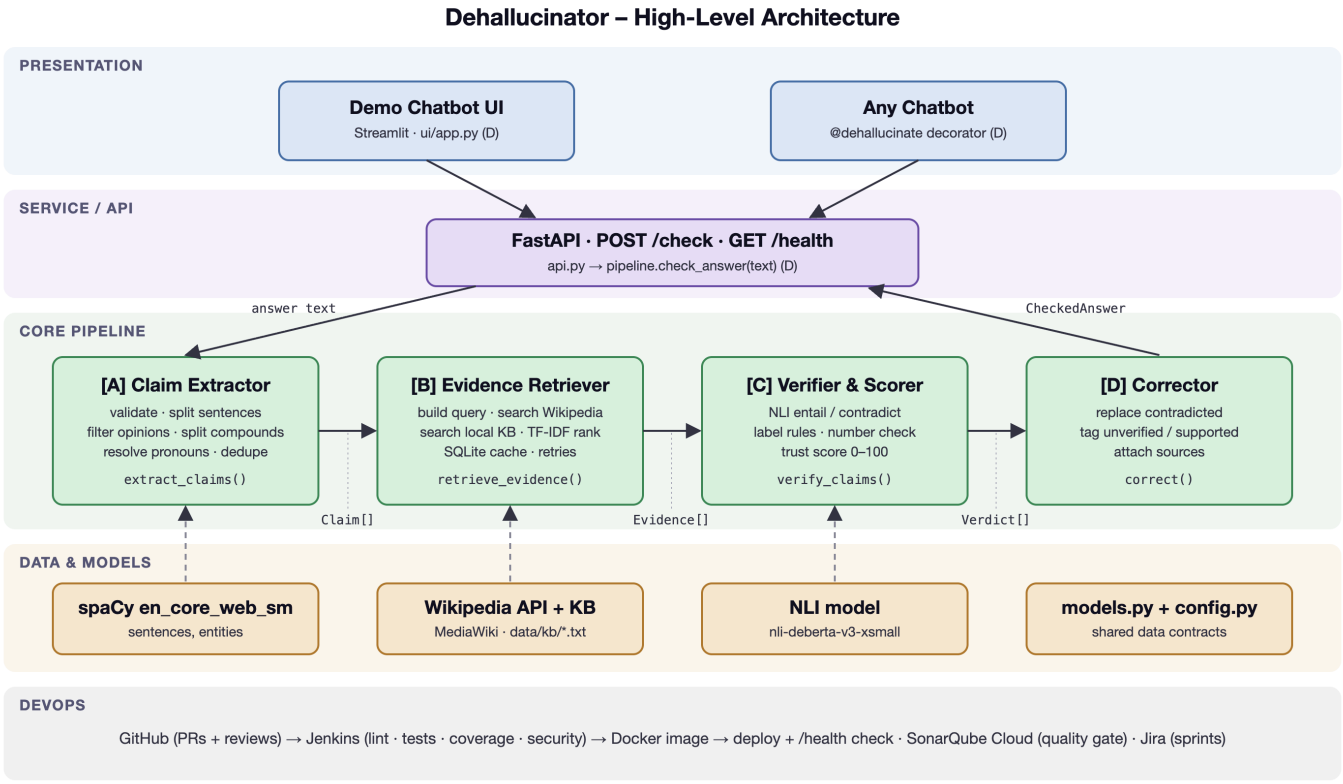


Figure 1: High-level architecture of Dehallucinator

1.2 Components

- **Claim Extractor (Role A):** splits the AI answer into standalone factual claims.
  Interface: `extract_claims(text) -> list[Claim]`.
- **Evidence Retriever (Role B):** finds supporting or contradicting text for each claim in Wikipedia and the local knowledge base.
  Interface: `retrieve_evidence(claims) -> list[Evidence]`.

- **Verifier & Scorer (Role C):** uses an NLI model to label each claim SUPPORTED, CONTRADICTED or NOT_ENOUGH_INFO, and computes a trust score.
  Interface: `verify_claims(claims, evidence) -> list[Verdict]`.
  See Architecture — Verifier & Scorer for the UML class and sequence diagrams.
- **Corrector & Integrator (Role D):** rewrites the answer using the verdicts and exposes the whole pipeline through the API, UI and decorator.
  Interface: `check_answer(text) -> CheckedAnswer`.

1.3 Data Flow

1. A user or chatbot sends an AI-generated answer to `POST /check`.
2. The API validates the input and calls `check_answer()`.
3. The Claim Extractor returns a list of `Claim` objects (id, text, sentence_index).
4. The Evidence Retriever returns `Evidence` objects (claim_id, source, URL, snippet, relevance) for each claim.
5. The Verifier returns one `Verdict` per claim (label, confidence score, best evidence) and an overall trust score.
6. The Corrector builds the corrected answer: contradicted sentences are replaced with the evidence and its source, unverified sentences are flagged, and supported sentences are tagged with their source.
7. The API returns a `CheckedAnswer` (original text, corrected text, claims, verdicts, trust score) as JSON, and the UI displays it with colour-coded claims.

1.4 Key Design Decisions

- **Fixed data contracts:** all modules exchange the same Pydantic models defined in `models.py`, so the four modules could be developed in parallel and tested independently.
- **Single configuration file:** all thresholds and limits live in `config.py`, so behaviour can be tuned without changing code.
- **Offline fallback:** evidence comes from both Wikipedia and a local knowledge base, so the system still works if Wikipedia is unreachable.
- **Load-once models:** the spaCy and NLI models are loaded on first use and reused, keeping each request fast.
- **No paid services:** all models and data sources are free and run locally, so the system has no API keys or usage costs.

1.5 Deployment View

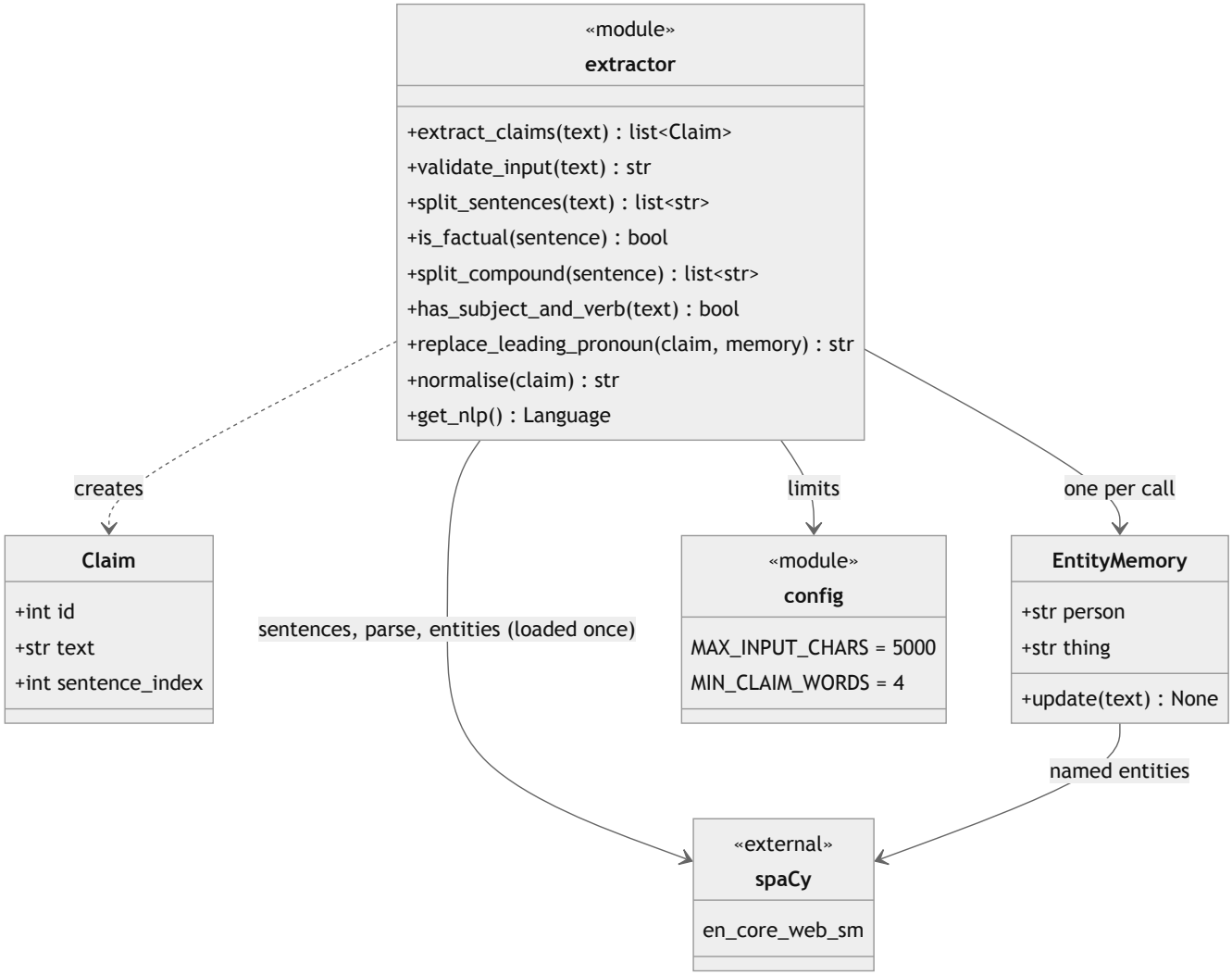
Code is hosted on GitHub, and every change goes through a reviewed pull request. Jenkins builds each branch and runs linting, unit tests, coverage and security scans. On the main branch it also builds a Docker image and deploys it, then checks `/health`. SonarQube Cloud analyses every pull request for code quality. Sprints and tickets are tracked in Jira.

2. Claim Extractor: Design (Role A – Sai Nikitha Perumalla)

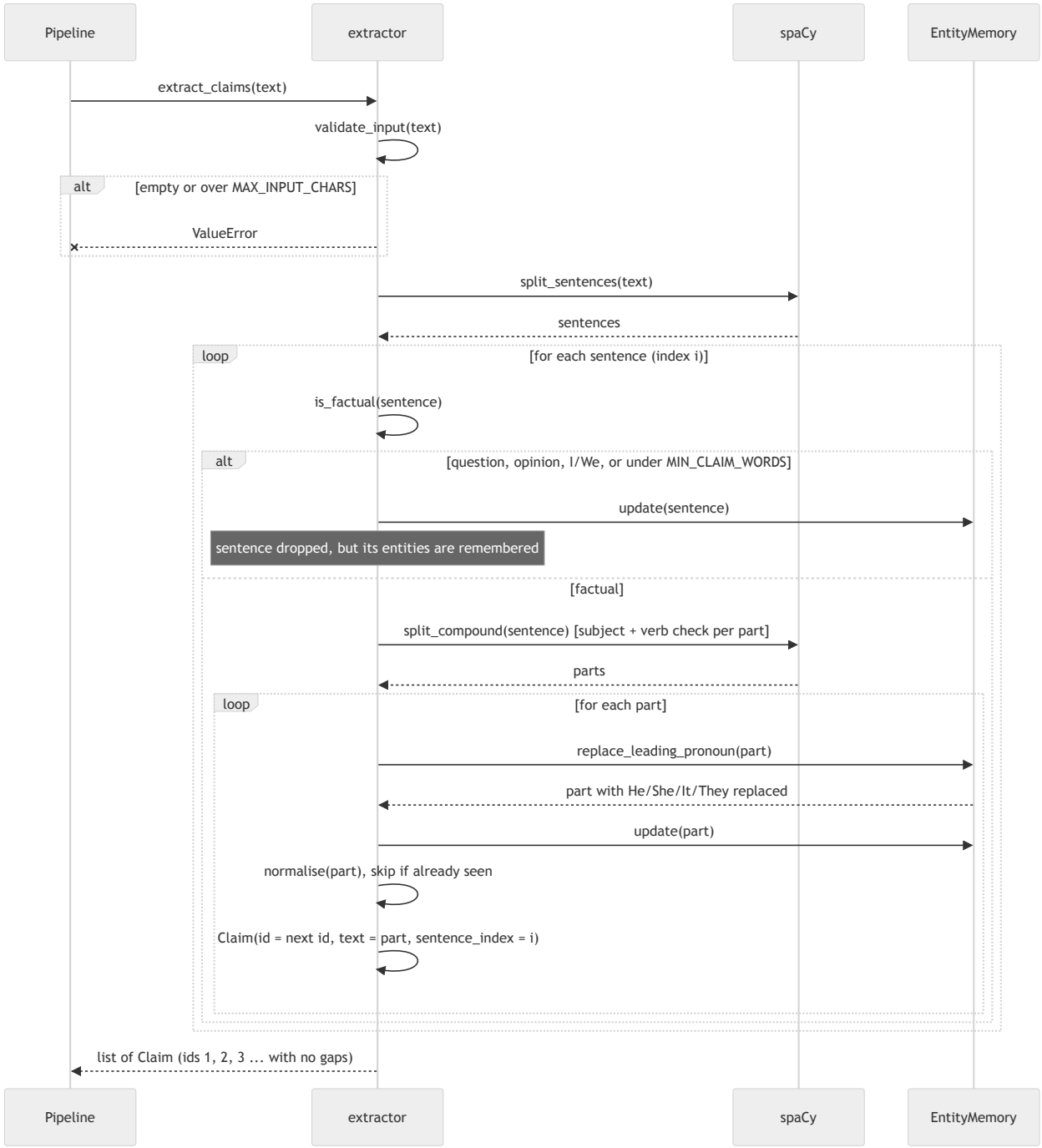
Module: `src/dehallucinator/claims/extractor.py`. Requirements covered: FR-1.1 to FR-1.7.

Class diagram

The extractor is mostly functions in one module, so the module is shown as a class with the `<<module>>` stereotype. `EntityMemory` is a real class: it remembers the last person and the last thing named, for pronoun replacement.



Sequence diagram: extract_claims()



Design decisions

Decision	Reason
spaCy model loaded once with `lru_cache`	Loading takes about a second; the corrector reuses the same model, so sentence numbers match.
A compound sentence is split only when both parts have a subject and a verb	Keeps lists like "Salt and pepper are spices." as one claim (FR-1.4).
The subject of a sentence wins over other entities in `EntityMemory`	In "The Eiffel Tower is in Paris. It is 330 m tall.", "It" means the tower, not Paris (FR-1.5).
He/She look up people only; It/They look up things only	In "Apple was founded by Steve Jobs.", "It" is Apple and "He" is Steve Jobs.

Decision	Reason
De-duplication runs after pronoun replacement	"It is in Paris." is caught as a repeat of "The Eiffel Tower is in Paris." (FR-1.6).
`sentence_index` is the original sentence, even for split claims	The corrector uses it to find where to put the tag in the answer.

3. Evidence Retriever: Architecture and Design (Role B – Shreya Raghuraj)

1. Purpose

Finds trustworthy evidence for each claim by searching Wikipedia, fetching page text, and returning the most relevant snippets. Requirements: FR-2.1 to FR-2.5.

2. Responsibilities

Step	Responsibility	Module	Requirement
1	Build a search query from a claim (entities first, then noun chunks, stop words removed, max 8 words)	`evidence/retriever.py` `build_query`	FR-2.1
2	Search Wikipedia (MediaWiki API, top 3 pages)	`evidence/retriever.py` `search_wikipedia`	FR-2.2
3	Fetch page text (extracts API, first 6000 chars, correct page URL)	`evidence/retriever.py` `fetch_page_text`	FR-2.3
4	Split text into snippets (sentence + next sentence) and rank by TF-IDF cosine similarity	`evidence/ranker.py` `rank_snippets`	FR-2.5
5	Orchestrate steps 1-4 per claim and return `Evidence` objects (pending)	`evidence/retriever.py` `retrieve_evidence`	FR-2.5

Not in scope: extracting claims (Role A), verifying claims against evidence (Role C), and the API/UI (Role D).

3. Interfaces

Input: `list[Claim]` from the Claim Extractor. **Output:** `list[Evidence]` for the Verifier.

Function	Input	Output
`build_query(claim_text)`	`str`	`str` (at most `MAX_QUERY_WORDS` words, `""` if empty)
`search_wikipedia(query)`	`str`	`list[str]` page titles (at most `WIKI_TOP_PAGES`; `[]` on error)
`fetch_page_text(title)`	`str`	`dict` with `title`, `url`, `text` (at most `WIKI_MAX_CHARS`), or `None`
`rank_snippets(claim_text, pages, top_n)`	`str`, `list[dict]`	`list[dict]` with `text`, `title`, `url`, `relevance` (0 to 1), best first
`retrieve_evidence(claims)`	`list[Claim]`	`list[Evidence]`

Data flow: `Claim -> build_query -> search_wikipedia -> fetch_page_text (per title) -> rank_snippets -> Evidence`

4. Dependencies

Dependency	Used for
Wikipedia MediaWiki API (`en.wikipedia.org/w/api.php`)	search and page text; requires a User-Agent header

Dependency	Used for
`requests`	HTTP calls (timeout `HTTP_TIMEOUT_S`)
`spaCy` + `en_core_web_sm`	entities and noun chunks for query building
`scikit-learn`	TF-IDF vectorizer and cosine similarity
`dehallucinator.models`	`Claim` and `Evidence` data classes
`dehallucinator.config`	`WIKI_TOP_PAGES`, `EVIDENCE_PER_CLAIM`, `HTTP_TIMEOUT_S`, `MAX_QUERY_WORDS`, `WIKI_MAX_CHARS`, `WIKI_API_URL`, `USER_AGENT`

5. Error handling

- Network and HTTP errors: `search_wikipedia` returns `[]` and `fetch_page_text` returns `None`, so a claim gets no evidence instead of crashing the pipeline.
- Empty claim text: `build_query` returns `""`.
- No relevant snippet: `rank_snippets` returns `[]`.

6. Testing

Unit tests in `tests/unit/test_retriever.py` and `tests/unit/test_ranker.py` (TC-2-01 to TC-2-05). All HTTP calls are mocked.

7. Local KB, cache and fallback (planned: FR-2.4, FR-2.6 to FR-2.8)

- **Local KB (FR-2.4):** all `data/kb/*.txt` files are read once at start-up and treated as pages with source `"local_kb"`. Their snippets are ranked together with Wikipedia snippets.
- **Cache (FR-2.6):** Wikipedia pages are stored in SQLite and reused if under `CACHE_TTL_DAYS` (7) old. Queries are parameterised so SQL-like input cannot break it.
- **Failure handling (FR-2.7, NFR-2):** on timeout or network error the retriever retries `HTTP_RETRIES` (2) times with a `HTTP_TIMEOUT_S` (5 s) timeout, logs a warning and continues with the local KB only.
- **No evidence (FR-2.8):** no `Evidence` is returned for a claim with no usable snippet.

8. Output object

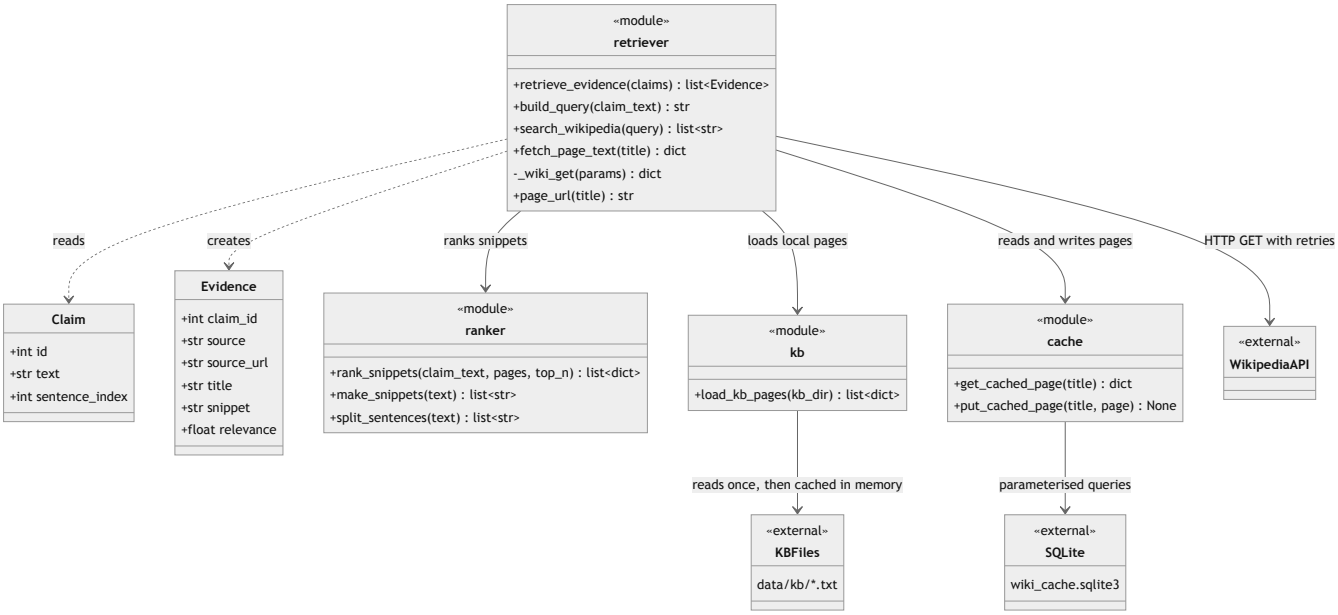
`Evidence = {claim_id, source, source_url, title, snippet, relevance}`, where `source` is `"wikipedia"` or `"local_kb"` and `relevance` is the TF-IDF cosine score (0 to 1). `rank_snippets` returns `text`, `title`, `url` and `relevance`; `retrieve_evidence` maps these to the `Evidence` fields.

Evidence Retriever: Design (DH-189)

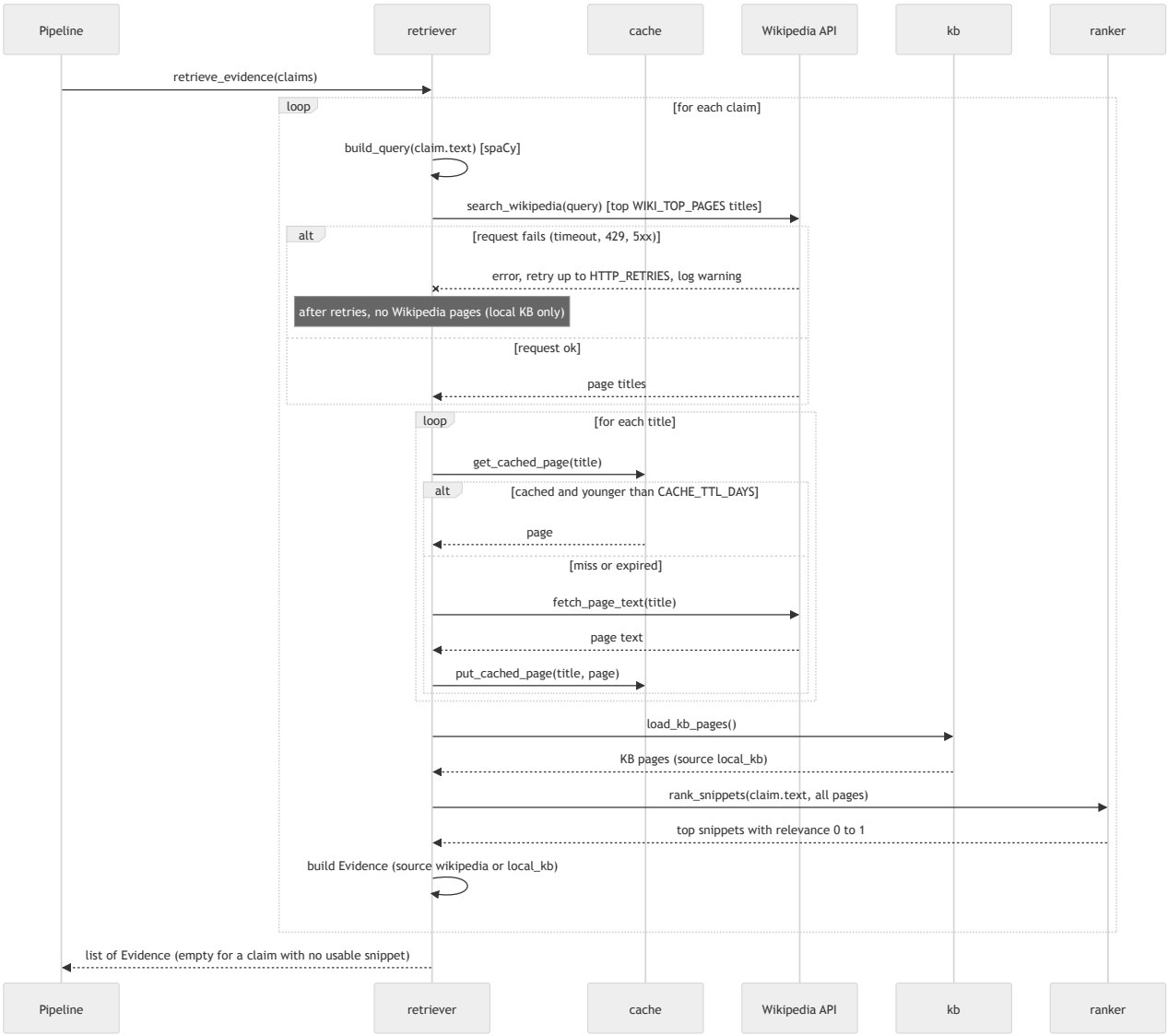
Module: `src/dehallucinator/evidence/` (`retriever.py`, `ranker.py`, `kb.py`, `cache.py`). Requirements covered: FR-2.1 to FR-2.8.

Class diagram

The retriever is built from functions grouped in modules, so each module is shown as a class with the `<<module>>` stereotype.



Sequence diagram: retrieve_evidence()



4. Verifier & Scorer: Architecture and Design (Role C – Ralph D'souza)

Project 39: Dehallucination Add-On Module ("Dehallucinator")

Component owner: Role C — Verifier & Scorer

1. Purpose

The Verifier & Scorer evaluates each extracted factual claim against the evidence returned by the Evidence Retriever. It assigns a verdict to each claim, selects evidence for that verdict, and calculates an overall trust score for the answer.

2. Position in the Pipeline

The Verifier & Scorer is the third stage of the core pipeline:

Claim Extractor → Evidence Retriever → Verifier & Scorer → Corrector

It receives `Claim` objects and `Evidence` objects, returns a list of `Verdict` objects, and provides the trust score used by the pipeline.

3. Interfaces

- `verify_claims(claims, evidence) -> list[Verdict]`: evaluates each claim against its available evidence.
- `trust_score(verdicts) -> float`: calculates an overall trust score between 0 and 100.

Implementation: `src/dehallucinator/verify/verifier.py`. The model name and thresholds are configured in `src/dehallucinator/config.py`.

4. Models and Data Contracts

The component uses the shared Pydantic models in `src/dehallucinator/models.py`.

- **Claim:** `id`, `text`, and `sentence_index`.
- **Evidence:** `claim_id`, `source`, `source_url`, `title`, `snippet`, and `relevance`.
- **Verdict:** `claim_id`, `label`, `score`, and `best_evidence`.

The verdict label is `SUPPORTED`, `CONTRADICTED`, or `NOT_ENOUGH_INFO`. The score is between 0 and 1. `best_evidence` may be `None` when no evidence is available.

5. Verification Process

5.1 Evidence Matching

For each claim, the verifier selects evidence items whose `claim_id` matches the claim ID. If no matching evidence exists, it returns `NOT_ENOUGH_INFO`, with a score of `0.0` and no best evidence.

5.2 Natural Language Inference

The verifier uses the `cross-encoder/nli-deberta-v3-xsmall` Natural Language Inference (NLI) model. It evaluates each evidence snippet against the corresponding claim text.

The tokenizer and model are loaded lazily on first use and cached in module-level variables. Predictions run with PyTorch gradient calculation disabled, and model logits are converted to probabilities using softmax.

5.3 Verdict Assignment

The configurable thresholds are:

- `SUPPORT_THRESHOLD = 0.70`
- `CONTRADICT_THRESHOLD = 0.70`

The decision rules are:

1. **SUPPORTED:** the highest entailment probability meets the support threshold and is at least as high as the highest contradiction probability.
2. **CONTRADICTED:** the highest contradiction probability meets the contradiction threshold, and the support rule has not been met.
3. **NOT_ENOUGH_INFO:** neither rule applies.

For `SUPPORTED` and `CONTRADICTED`, the score is the corresponding maximum probability. For `NOT_ENOUGH_INFO` when evidence exists, the score is `1 - max(max_entailment, max_contradiction)`. The selected best evidence comes from the corresponding prediction.

6. Trust Scoring

The scorer maps verdicts to values:

Verdict	Value
`SUPPORTED`	1.0
`NOT_ENOUGH_INFO`	0.5
`CONTRADICTED`	0.0

It averages these values, multiplies the result by 100, and rounds to one decimal place. If there are no verdicts, the trust score is `100.0`.

7. Runtime Considerations

- The model and tokenizer load only when verification first requires them.
- Once loaded, they are reused for subsequent calls in the same process.
- Inference runs without gradient calculation.
- Claims without matching evidence receive `NOT_ENOUGH_INFO` without an NLI prediction for that claim.

8. Dependencies and Configuration

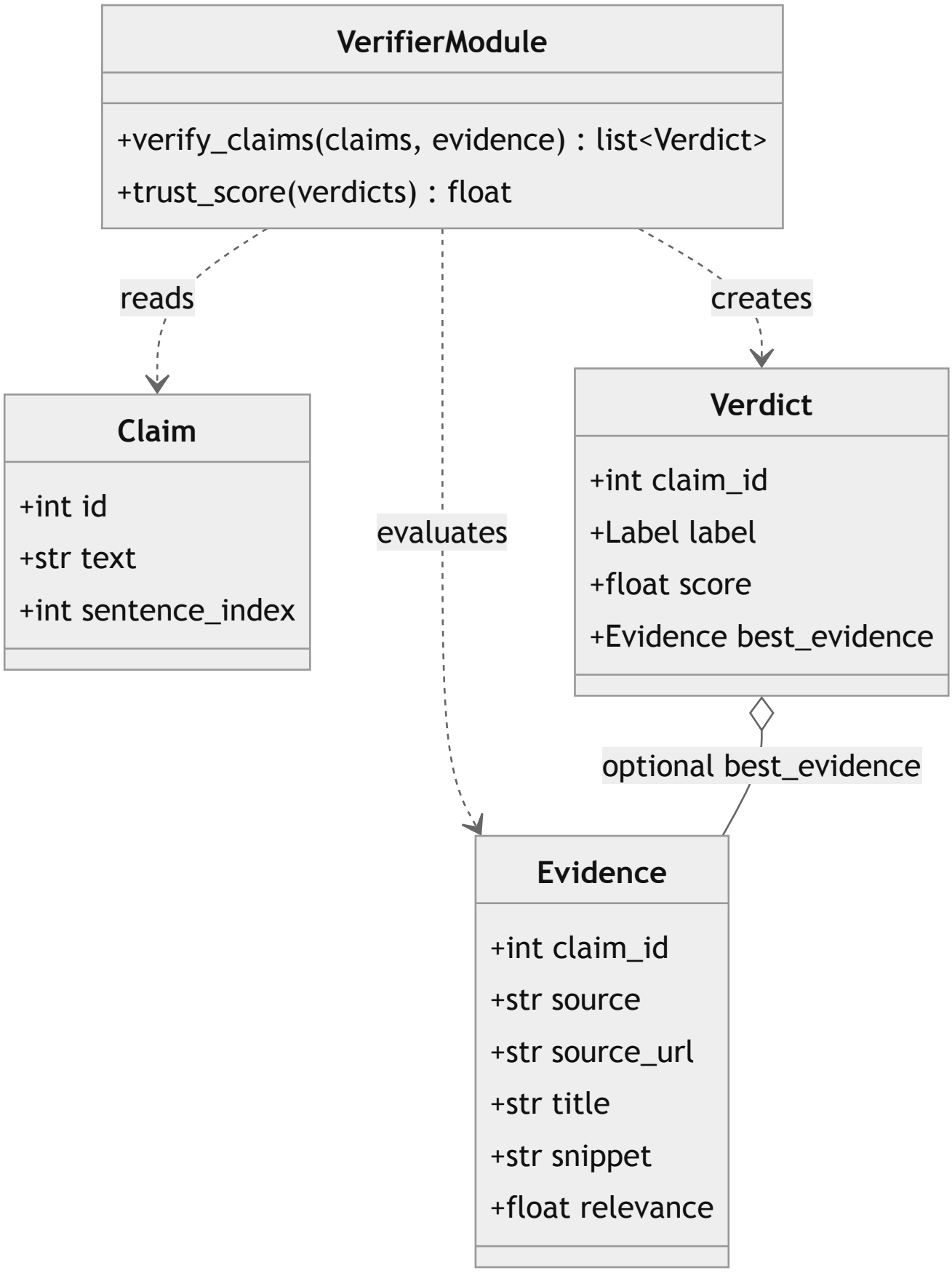
- `transformers` provides the tokenizer and sequence-classification model.
- `torch` performs inference and probability calculation.
- `src/dehallucinator/models.py` defines the shared data contracts.
- `src/dehallucinator/config.py` defines the NLI model identifier and thresholds.

9. Output to the Next Stage

The verifier returns one `Verdict` per claim, including its ID, label, score, and best evidence where available. The Corrector uses these verdicts and their evidence to decide whether to preserve a sentence, attach a source, flag uncertainty, or replace contradicted content.

10. UML Class Diagram

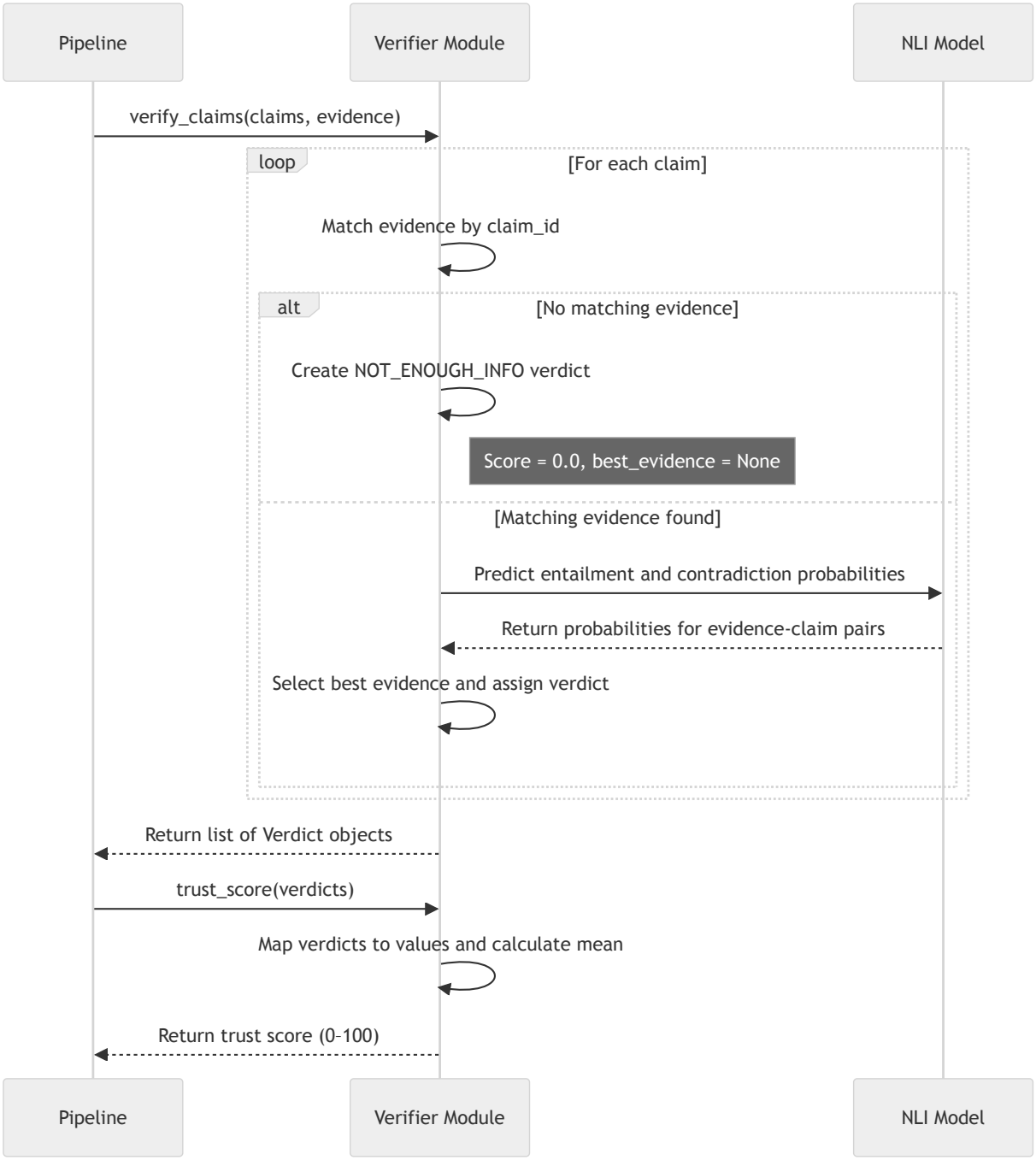
The diagram shows the shared data models used by the verifier and the relationships between the module-level verification functions and those models.



Note: `VerifierModule` represents the module-level functions in `verifier.py`, not a Python class.

11. UML Sequence Diagram

The sequence diagram illustrates how `verify_claims()` matches evidence to each claim, handles claims with no matching evidence, obtains NLI predictions, assigns verdicts, and returns the results to the pipeline.



5. Corrector, Pipeline, API, UI and Deployment (Role D – Satwik)

Owner: Satwik (Role D). Section 4 of the Architecture Document.

1. Purpose

Connects the four pipeline stages into one call and makes it usable from outside: a REST API for any application, a decorator for Python chatbots, and a demo UI for end users. Also covers how the system is built, tested and deployed. Requirements: FR-4.1 to FR-4.6, NFR-3, NFR-5.

2. Components

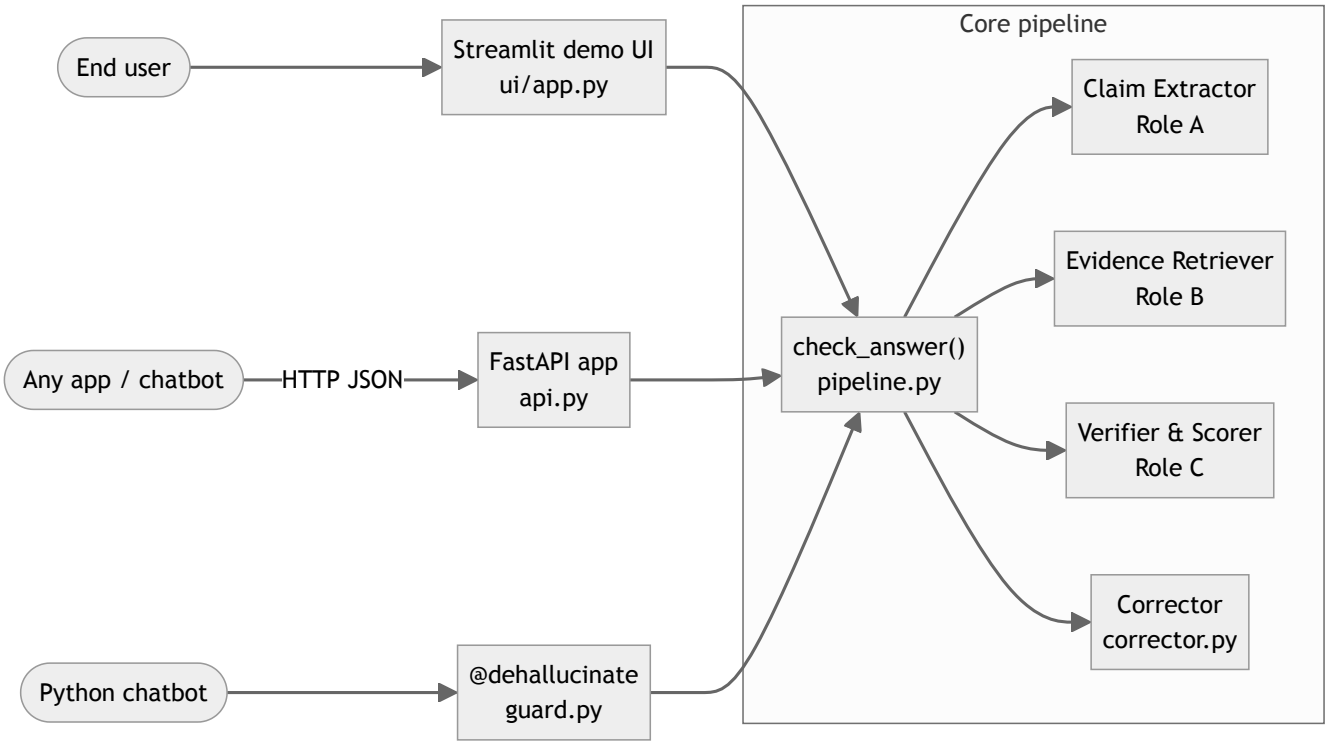


Figure 4.1: Entry points (UI, API, decorator) and how they reach the core pipeline.

Component	Module	Responsibility	Requirement	Status
Corrector	`correct/corrector.py` `correct`	Rewrites the answer from the verdicts: CONTRADICTED claims replaced by the evidence and its source, NOT_ENOUGH_INFO tagged `[unverified]`, SUPPORTED tagged with the source	FR-4.1	Sprint 1
Pipeline	`pipeline.py` `check_answer`	Runs extract → retrieve → verify → score → correct, logs each stage's time, returns a `CheckedAnswer`	FR-4.2	Sprint 1
REST API	`api.py` `app`	`POST /check` and `GET /health`; validates input; maps errors to HTTP status codes	FR-4.4, NFR-3	Sprint 1
Decorator	`guard.py` `dehallucinate`	Wraps any function that returns a `str` so it returns a `CheckedAnswer`	FR-4.3	Sprint 2 (DH-178)

Component	Module	Responsibility	Requirement	Status
Demo UI	`ui/app.py`	Sample drop-down and paste box; colour-coded claims, sources, trust score and corrected answer	FR-4.5, FR-4.6	Sprint 2 (DH-179)

The API, the UI and the decorator all call the same `check_answer()`, so every entry point gives the same result and the pipeline is tested once.

3. Interfaces

Pipeline: `check_answer(text: str) -> CheckedAnswer`, where `CheckedAnswer = {original, corrected, claims, verdicts, trust_score}` (defined in `models.py`). Raises `ValueError` for empty input or input over `MAX_INPUT_CHARS` (5000).

Corrector: `correct(text: str, claims: list[Claim], verdicts: list[Verdict]) -> str`. It splits sentences with the Claim Extractor's own spaCy model and skips whitespace-only sentences in the same way, so a claim's `sentence_index` points at the same sentence in both modules.

REST API (served with `uvicorn dehallucinator.api:app`):

Endpoint	Request	Responses
`POST /check`	`{"text": "<AI answer>"}`; 1 to 5000 characters, not blank	200: `CheckedAnswer` JSON. 422: invalid or malformed input. 500: `{"detail": "Internal server error"}`
`GET /health`	none	200: `{"status": "ok"}`

Request flow for `POST /check`:

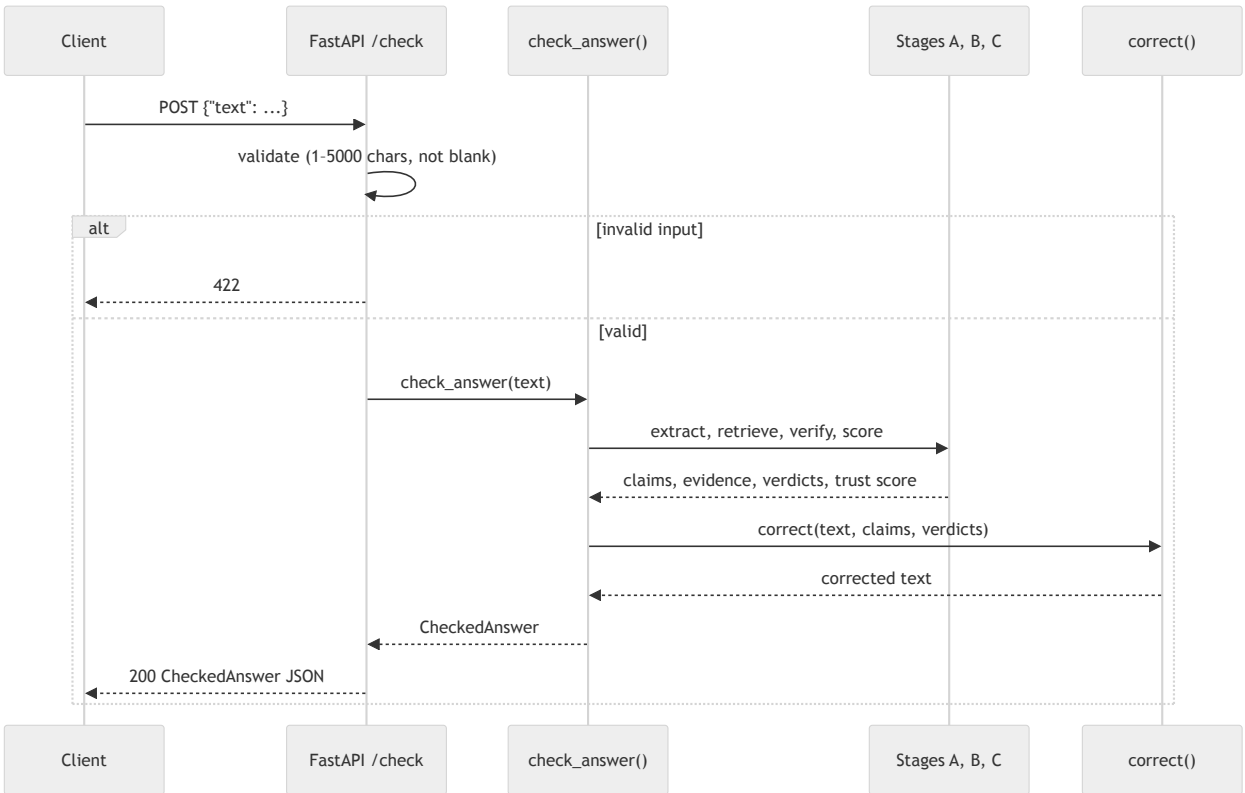


Figure 4.2: Request flow for `POST /check`.

4. Dependencies

Dependency	Used for
FastAPI + pydantic	REST API, request validation, JSON responses from the shared models
uvicorn	ASGI server that runs the API
Streamlit	Demo UI
spaCy `en_core_web_sm`	Sentence spans in the Corrector (shared with the Claim Extractor)
Modules A, B, C	`extract_claims`, `retrieve_evidence`, `verify_claims`, `trust_score`
`dehallucinator.config`	`MAX_INPUT_CHARS`

5. Error handling

Situation	Behaviour
Missing, empty, blank, wrong-type or over-long `text`, or malformed JSON	422; the pipeline is not called
`ValueError` from the pipeline	422 with the error message
Any other exception	Logged with its stack trace on the server; the client gets 500 with a generic message and no stack trace (NFR-3)
No factual claims	Retriever and Verifier are skipped; the corrected answer equals the original and the trust score is 100
No evidence for a claim	The Verifier returns NOT_ENOUGH_INFO (FR-3.4) and the Corrector tags the claim `[unverified]`

The `/check` endpoint is a plain `def`, so FastAPI runs the CPU-heavy pipeline in a worker thread and `/health` stays responsive while a check is running.

6. Deployment view

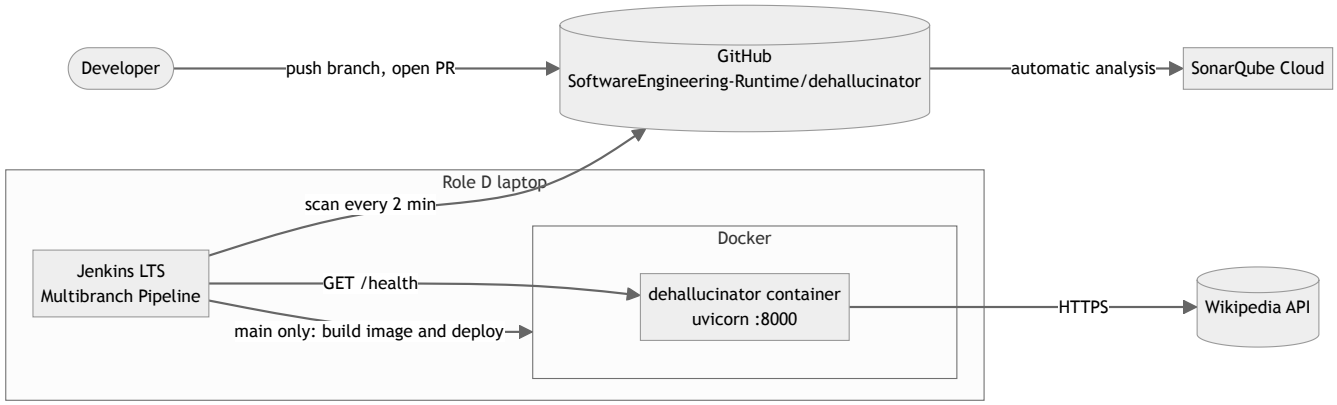


Figure 4.3: Deployment view. Jenkins and the container run on Role D's laptop.

- **Source control:** GitHub. `main` is protected; every change is a pull request with one approval. Commits start with the Jira key so they appear on the ticket.
- **CI (Jenkins):** a Multibranch Pipeline job scans the repository every 2 minutes and builds every branch that has a `Jenkinsfile`. The 8 stages (DH-153, DH-180, DH-182, manual 11.4): Checkout, Setup, Lint (ruff), Security scan (bandit High issues, pip-audit), Unit tests (JUnit report, HTML coverage report, build

fails below 80% coverage), Integration tests, and on `main` only Build image and Deploy (replace the container, then wait for `/health` with `ci/wait_for_health.py`). If Docker is missing on the agent, the two image stages mark the build UNSTABLE instead of failing it.

- **Container (DH-181):** a `python:3.11-slim` image with the dependencies (CPU-only PyTorch), the spaCy and NLI models and the API, running as a non-root user and started with one command, `docker run -p 8000:8000 dehallucinator` (NFR-5). Docker's `HEALTHCHECK` calls `GET /health`. The same image can start the Streamlit UI on port 8501.
- **Code quality:** SonarQube Cloud analyses every push (DH-154, DH-183).
- **External service:** the running container needs HTTPS access to Wikipedia; without it the Retriever falls back to the local KB baked into the image (FR-2.7).

7. Testing

Level	Tests	Notes
Unit	`tests/unit/test_corrector.py` (TC-4-01 to TC-4-05), `test_pipeline.py`, `test_api.py` (TC-4-06 to TC-4-09), `test_guard.py` (TC-4-10), `test_ui.py`	Stages and `check_answer` are faked, so no network or model is used
Integration	`tests/integration/test_pipeline.py` (Role C, DH-197)	Full `check_answer` with mocks
System	`tests/system/test_end_to_end.py` (DH-198)	API with the 10 sample answers
Security	`tests/security/test_security.py` (DH-199)	Input limits, malformed JSON, script input

Software Design: Corrector, Pipeline and API (Role D)

Owner: Satwik (Role D). Detailed design for the modules described at a high level in Architecture Section 4. Requirements: FR-4.1 to FR-4.6.

1. Modules

Module	Public interface	Requirement
`correct/corrector.py`	`correct(text, claims, verdicts) -> str`	FR-4.1
`pipeline.py`	`check_answer(text) -> CheckedAnswer`	FR-4.2
`guard.py`	`@dehallucinate`	FR-4.3
`api.py`	FastAPI `app`: `POST /check`, `GET /health`	FR-4.4
`ui_render.py`, `ui/app.py`	Streamlit page and its HTML helpers	FR-4.5, FR-4.6

2. Class diagram

The shared models come from `models.py` (frozen in manual Section 4.2). Role D adds the private `_Edit` record in the Corrector, the `CheckRequest` body in the API and the `ClaimRow` view model in the UI.

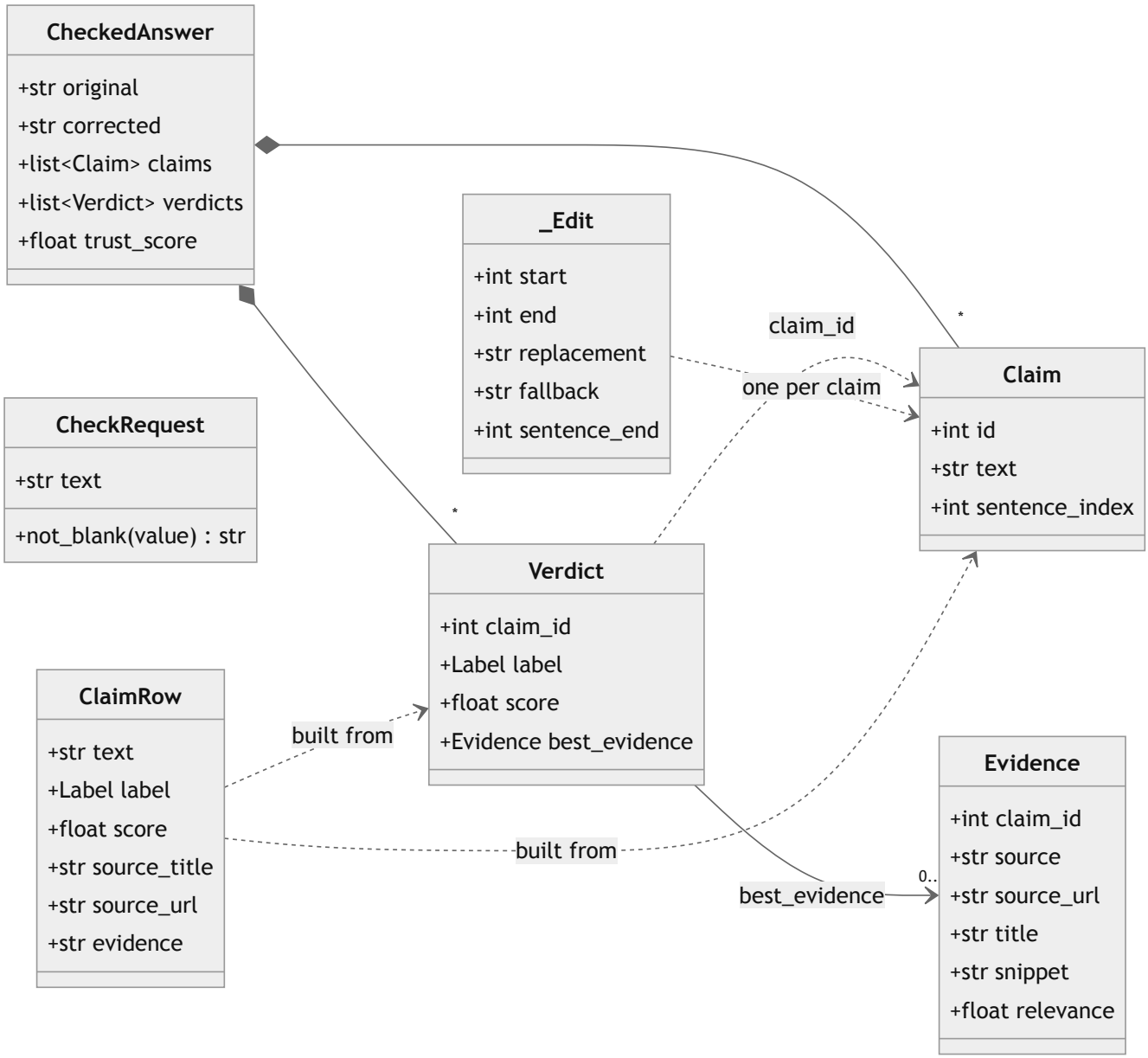


Figure D.1: Data classes used by the Role D modules.

3. Corrector design (FR-4.1)

`correct()` never edits the string in place. It first works out one `_Edit` per claim (a character range to replace, or an insertion point), then applies all edits in one pass from left to right. This keeps character positions valid however many claims there are.

Steps

1. If there are no claims, return the text unchanged.
2. Split the text into sentence spans with the Claim Extractor's own spaCy model (`get_nlp()`), skipping whitespace-only sentences and trimming surrounding whitespace, so `sentence_index` means the same sentence in both modules.
3. For each claim, in id order, decide its edit (Figure D.2).
4. Apply the edits: overlapping replacements become insertions at the end of their sentence (the `fallback` text), and no insertion may land inside replaced text.

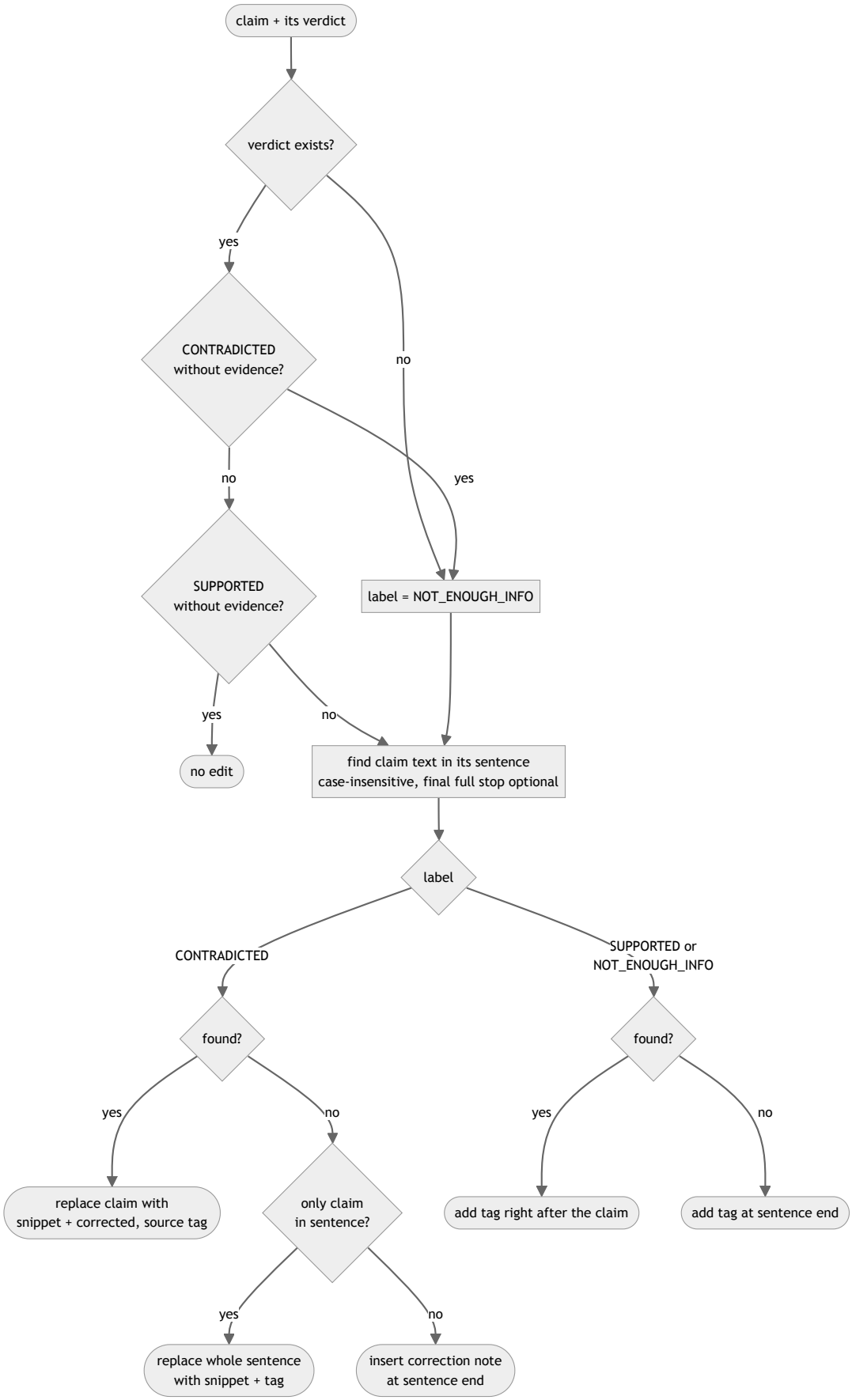


Figure D.2: How `_edit_for()` chooses the edit for one claim.

Label	Tag added	Example output
CONTRADICTED (with evidence)	`[corrected, source: <url>]` after the snippet	`The tower was built from 1887 to 1889. [corrected, source: https://en.wikipedia.org/wiki/Eiffel_Tower]`
NOT_ENOUGH_INFO	`[unverified]`	`The Eiffel Tower has 1,710 steps. [unverified]`
SUPPORTED	`[source: <url>]`	`The Eiffel Tower is located in Paris. [source: https://en.wikipedia.org/wiki/Eiffel_Tower]`

Evidence snippets are collapsed to one line (`" ".join(snippet.split())`) so a snippet with line breaks does not break the paragraph layout.

4. Pipeline design (FR-4.2)

`check_answer()` calls each stage through the `_timed()` helper, which records the stage's duration in milliseconds. One INFO log line per answer lists the claim count, trust score and all stage times. Exceptions are not caught here; the caller (API, UI or decorator) decides how to report them.

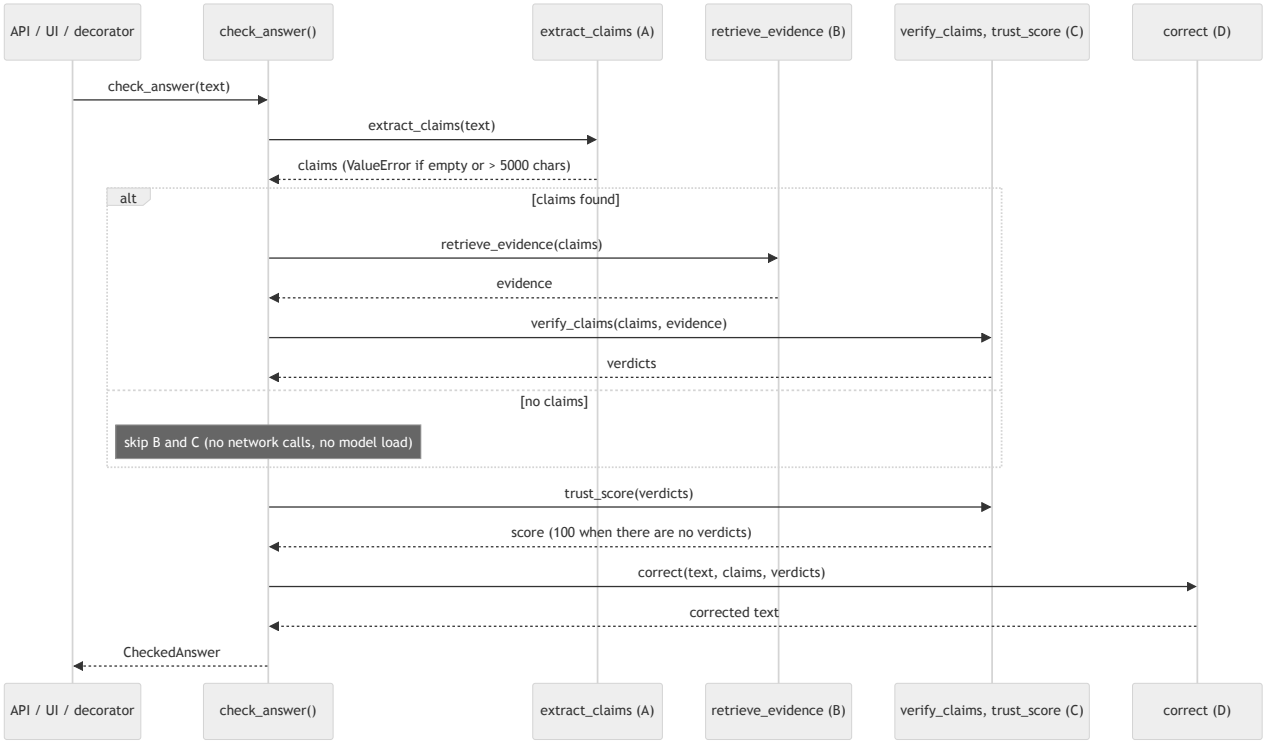


Figure D.3: `check_answer()` sequence.

5. API specification (FR-4.4)

The machine-readable spec is `openapi.json` (OpenAPI 3.1), generated from the FastAPI app. `tests/unit/test_api.py::test_committed_openapi_spec_matches_the_app` fails if the file and the code ever differ. When the app is running, the same spec is served at `/openapi.json` and as interactive docs at `/docs`.

`GET /health`

Purpose	Liveness check for Docker and the Jenkins Deploy stage
Request	No body

200	`{"status": "ok"}`

`POST /check`

Purpose	Check and correct one AI answer
Request body	`CheckRequest`: `{"text": string}`, 1 to 5000 characters, not only whitespace
200	`CheckedAnswer`
422	Invalid body or malformed JSON: `{"detail": [ {loc, msg, type}, ... ]}`. `ValueError` from the pipeline: `{"detail": "<message>"}`
500	`{"detail": "Internal server error"}`; the stack trace is logged on the server only

Schemas

Schema	Fields
`CheckRequest`	`text: string` (minLength 1, maxLength 5000)
`CheckedAnswer`	`original: string`, `corrected: string`, `claims: Claim[]`, `verdicts: Verdict[]`, `trust_score: number` (0 to 100)
`Claim`	`id: integer`, `text: string`, `sentence_index: integer`
`Verdict`	`claim_id: integer`, `label: "SUPPORTED" | "CONTRADICTED" | "NOT_ENOUGH_INFO"`, `score: number` (0 to 1), `best_evidence: Evidence | null`
`Evidence`	`claim_id: integer`, `source: "wikipedia" | "local_kb"`, `source_url: string`, `title: string`, `snippet: string`, `relevance: number` (0 to 1)

Error mapping inside the endpoint

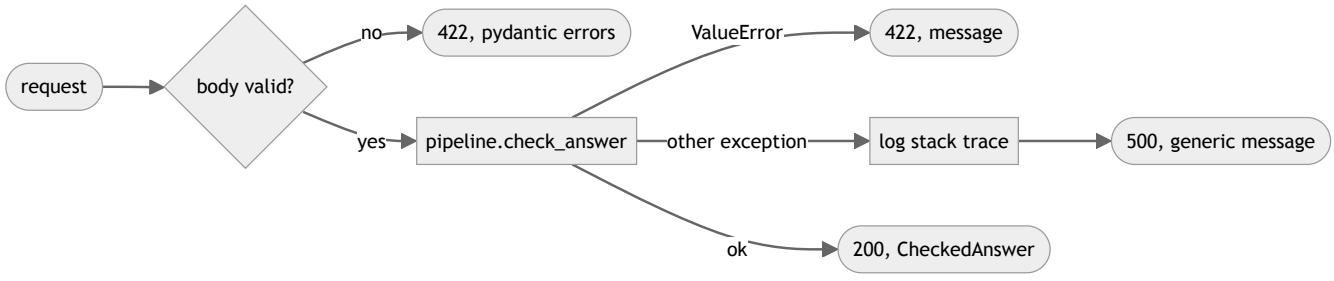


Figure D.4: Status codes returned by `POST /check`.

The endpoint is a plain `def`, so FastAPI runs it in a worker thread and a slow check does not block `/health`.

6. Decorator design (FR-4.3)

`dehallucinate(func)` returns a wrapper built with `functools.wraps`, so the name, docstring and `__wrapped__` of the original function are kept. The wrapper calls `func`, raises `TypeError` if the result is not a `str` (before calling the pipeline), and otherwise returns `pipeline.check_answer(result)`. It is typed with `ParamSpec`, so editors still show the original function's parameters, and it also works on methods.

7. Demo UI design (FR-4.5, FR-4.6)

`ui/app.py` is a thin Streamlit script; the display logic lives in `ui_render.py` so it can be unit-tested without a browser.

Function	Responsibility
`load_samples()`	Read and check `data/samples.json` (10 answers on KB topics, each ≤ 5000 chars)
`claim_rows(result)`	Join claims with their verdicts; a claim without a verdict is shown as unverified
`claim_html(row)`	Coloured card: green SUPPORTED, red CONTRADICTED, grey unverified; evidence and source
`source_html(title, url)`	Clickable link only for `http(s)://` URLs; `kb://` and anything else is plain text
`trust_colour(score)`	Green ≥ 75, grey 40 to 74.9, red < 40

Escaping rule (NFR-3, FR-4.6): every string that comes from the answer or the evidence passes through `html.escape` before it is placed in HTML, including the corrected answer. URLs are escaped with `quote=True` so a `"` cannot break out of the `href`. Only the fixed markup written in `ui_render.py` and `ui/app.py` is rendered as HTML.

8. Design decisions

Decision	Reason
Edits are collected first and applied in one pass	Positions stay valid when several claims are in one sentence or overlap
Corrector reuses the extractor's spaCy model and sentence rules	`sentence_index` must point at the same sentence in both modules; one model in memory
Skip retriever and verifier when there are no claims	A greeting gets an instant answer without network calls or loading the NLI model
Errors are mapped in the API, not in the pipeline	The library raises normal Python exceptions; HTTP codes are an API concern
OpenAPI file generated from the code and checked by a test	The documented API cannot drift from the real one
UI helpers separated from the Streamlit script	Escaping and colours are tested directly (`tests/unit/test_ui.py`)